CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 22: Independent Component Analysis and Component Visualization

Problem Statement: In a hospital ECG recording, the mother's heartbeat masks the foetal heartbeat.Doctors need the two signals separated (the cocktail-party problem). Three synthetic source signals (sine, square and sawtooth, or maternal and foetal ECG) mixed through a random mixing matrix. The objective is to separate mixed signals into independent sources with ICA. Use Scikit-learn, NumPy, Matplotlib.

Tasks:

1. Generate the source signals and mix them.

2. Apply FastICA to recover the sources.

3. Apply PCA to the same mixtures for comparison.

4. Plot original, mixed, ICA-recovered and PCA-recovered signals.

5. Measure recovery quality with correlation to the true sources.

FORMULA:

1. Source Signal Generation

$S(t) = [S1(t), S2(t), S3(t)]$

2. Mixing Model

$X = A \times S$

3. ICA Separation Model

S_recovered = W × X

4. Unmixing Matrix

W = A_inverse

5. FastICA Objective

Maximize Independence = E[G(w.T × X)]

6. PCA Transformation

X_PCA = X × Eigenvectors

7. Correlation Coefficient

r = Cov(X,Y) / (SD(X) × SD(Y))

8. Recovery Quality

Recovery Accuracy = |Correlation(S_original, S_recovered)| × 100

9. Mean Squared Error

MSE = Sum((S_original - S_recovered)²) / Number of Samples

10. Signal Mixing Matrix

X = A × S + Noise

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.signal import square, sawtooth

from scipy.optimize import linear_sum_assignment
```

```
from sklearn.decomposition import FastICA, PCA

from sklearn.preprocessing import StandardScaler

np.random.seed(42)

# 1. Generate three source signals

n_samples = 2000

t = np.linspace(0, 10, n_samples)

source1 = np.sin(2 * np.pi * 1 * t)          # Sine wave

source2 = square(2 * np.pi * 0.5 * t)        # Square wave

source3 = sawtooth(2 * np.pi * 0.3 * t)      # Sawtooth wave

# Combine sources into a matrix

S = np.c_[source1, source2, source3]

# Standardize source signals

S = StandardScaler().fit_transform(S)

print("Original source shape:", S.shape)

# 2. Mix the source signals using a random mixing matrix

A = np.array([

    [1.0, 0.5, 0.2],

    [0.3, 1.0, 0.5],

    [0.5, 0.2, 1.0]

])
```

```
X = S @ A.T

print("\nMixing matrix:")

print(A)

print("Mixed signal shape:", X.shape)

# 3. Apply FastICA to recover independent sources

ica = FastICA(

    n_components=3,

    whiten="unit-variance",

    random_state=42,

    max_iter=2000,

    tol=0.0001

)

S_ica = ica.fit_transform(X)

print("\nFastICA completed.")

# 4. Apply PCA to the same mixtures

pca = PCA(n_components=3)

S_pca = pca.fit_transform(X)

print("PCA completed.")

print("PCA explained variance ratio:",

      np.round(pca.explained_variance_ratio_, 4))
```

```
# 5. Plot original, mixed, ICA and PCA signals
signals = [
    (S, "Original Source Signals"),
    (X, "Mixed Signals"),
    (S_ica, "ICA Recovered Components"),
    (S_pca, "PCA Components")
]
fig, axes = plt.subplots(4, 3, figsize=(15, 10))
for row, (data, title) in enumerate(signals):
    for col in range(3):
        axes[row, col].plot(t, data[:, col])
        axes[row, col].set_title(
            f"{title} - Component {col + 1}"
        )
        axes[row, col].set_xlabel("Time")
        axes[row, col].set_ylabel("Amplitude")
        axes[row, col].grid(True)
plt.tight_layout()
plt.show()
# 6. Calculate correlations with true source signals
```

```
def correlation_matrix(true_sources, estimated_sources):
    result = np.zeros((true_sources.shape[1],
                       estimated_sources.shape[1]))
    for i in range(true_sources.shape[1]):
        for j in range(estimated_sources.shape[1]):
            result[i, j] = np.corrcoef(
                true_sources[:, i],
                estimated_sources[:, j]
            )[0, 1]
    return result
corr_ica = correlation_matrix(S, S_ica)
corr_pca = correlation_matrix(S, S_pca)
print("\nCorrelation: Original Sources vs ICA Components")
print(np.round(corr_ica, 3))
print("\nCorrelation: Original Sources vs PCA Components")
print(np.round(corr_pca, 3))
# Plot correlation heatmaps
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, corr, title in [
    (axes[0], corr_ica, "ICA Correlations"),
```

```
    (axes[1], corr_pca, "PCA Correlations")
]:
    image = ax.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xlabel("Estimated Component")
    ax.set_ylabel("Original Source")
    ax.set_xticks(range(3))
    ax.set_yticks(range(3))
    for i in range(3):
        for j in range(3):
            ax.text(j, i, f"{corr[i, j]:.2f}",
                    ha="center", va="center")
    fig.colorbar(image, ax=ax)
plt.tight_layout()
plt.show()
# 7. Calculate best source-to-component correlation matching
def best_matching_score(corr):
    # Find the component assignment maximizing absolute correlation
    rows, cols = linear_sum_assignment(-np.abs(corr))
    matched = np.abs(corr[rows, cols])
```

```
    return matched, rows, cols

ica_scores, rows_ica, cols_ica = best_matching_score(corr_ica)

pca_scores, rows_pca, cols_pca = best_matching_score(corr_pca)

print("\nRECOVERY QUALITY")

print("ICA matched absolute correlations:",

      np.round(ica_scores, 3))

print("ICA mean absolute correlation:",

      round(np.mean(ica_scores), 4))

print("\nPCA matched absolute correlations:",

      np.round(pca_scores, 3))

print("PCA mean absolute correlation:",

      round(np.mean(pca_scores), 4))

if np.mean(ica_scores) > np.mean(pca_scores):

    print("\nICA recovered the original sources more accurately.")

else:

    print("\nPCA had a higher matched correlation in this run.")

print("\nExperiment 22 completed successfully.")
```

OUTPUT:

```
Original source shape: (2000, 3)
```

Mixing matrix:

[[1. 0.5 0.2]

 [0.3 1. 0.5]

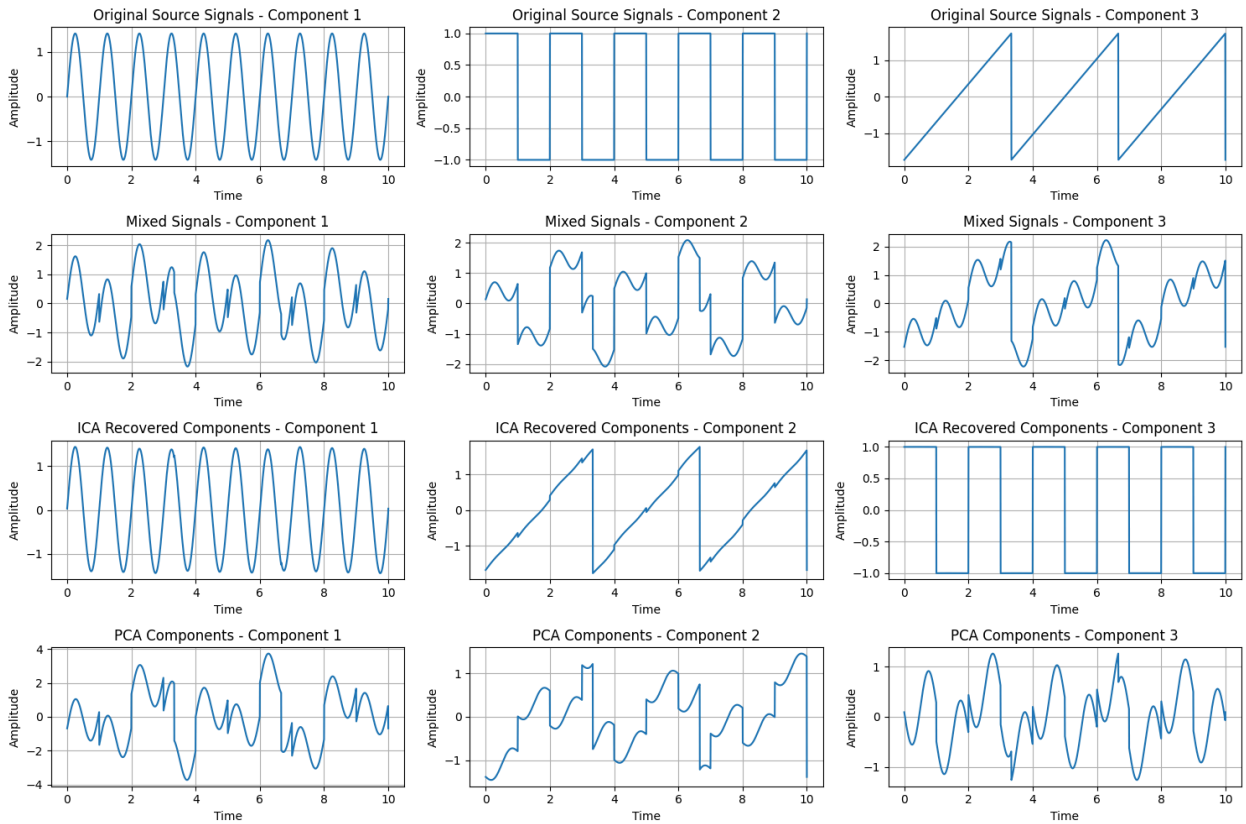
 [0.5 0.2 1.]]

Mixed signal shape: (2000, 3)

FastICA completed.

PCA completed.

PCA explained variance ratio: [0.7566 0.1374 0.1059]



Correlation: Original Sources vs ICA Components

[[1. 0.02 0.001]

 [-0.001 0. 1.]

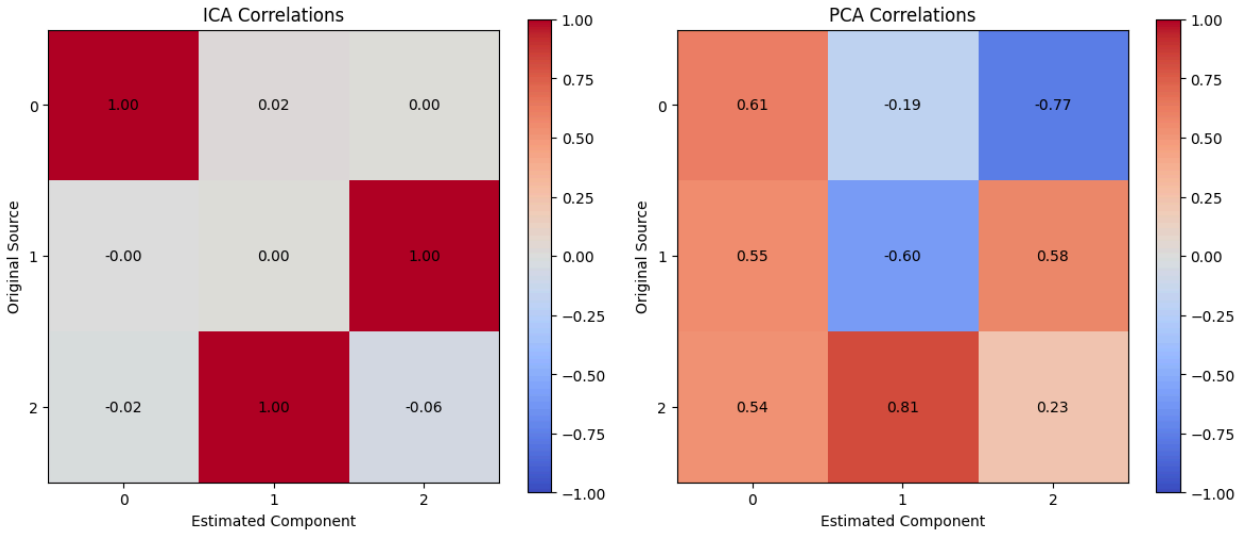
 [-0.019 0.998 -0.059]]

Correlation: Original Sources vs PCA Components

[[0.612 -0.188 -0.769]

 [0.549 -0.598 0.583]

 [0.536 0.813 0.227]]



RECOVERY QUALITY

ICA matched absolute correlations: [1. 1. 0.998]

ICA mean absolute correlation: 0.9993

PCA matched absolute correlations: [0.769 0.549 0.813]

PCA mean absolute correlation: 0.7102

ICA recovered the original sources more accurately.

Experiment 22 completed successfully.